In [ ]:

🔥17. Indexing & Slicing — Predict the Output

Each cell: **predict before running**. The patterns that bit you on the test are here.

17a. Step and reverse slicing

In [187…
```
# a[1:triples = [(l,h,scores[l,h]) for l in range(L) for h in range(H)]
#     return sorted(triples, key=lambda x: -x[2])6]
```

Out[187… array([20, 30, 40, 50, 60])

In [172…
```
a = np.array([10, 20, 30, 40, 50, 60])

print(a[::-1])        # reverse entire array
print(a[::2])         # every other element (0,2,4)
print(a[1::2])        # every other starting at index 1
print(a[-3:])         # last 3
print(a[:-2])         # all but last 2
print(a[-4:-1])       # index -4 up to (not including) -1
```

```
[60 50 40 30 20 10]
[10 30 50]
[20 40 60]
[40 50 60]
[10 20 30 40]
[30 40 50]
```

17b. 2D slicing — rows and columns

In [ ]:
```
A = np.arange(12).reshape(3, 4)
print("A =\n", A)
print()
print(A[:, ::-1])      # reverse columns — RIGHT to LEFT
print(A[::-1, :])      # reverse rows — BOTTOM to TOP
print(A[::-1, ::-1])   # flip both
print(A[:, -2:])       # last 2 columns
print(A[-2:, :])       # last 2 rows
print(A[1:, 1:])       # submatrix from (1,1)
```

17c. argsort + slice — the pattern that bit you

In [ ]:
```
a = np.array([3., 1., 4., 1., 5., 9., 2., 6.])

# argsort gives ASCENDING order
idx_asc = np.argsort(a)
print("ascending idx:", idx_asc)
```

```
print("ascending vals:", a[idx_asc])

# top-3 indices (ascending order of value — WRONG for most purposes)
print("top-3 wrong order:", a[idx_asc[-3:]])

# top-3 indices DESCENDING — add [::-1]
idx_top3 = idx_asc[-3:][::-1]
print("top-3 correct:    ", a[idx_top3])

# 2D version — per row
A = np.random.randn(4, 6)
idx_2d = np.argsort(A, axis=1)[:, -3:][:, ::-1]   # top-3 per row, descendin
vals_2d = np.take_along_axis(A, idx_2d, axis=1)
print("2D top-3 sorted descending per row:", np.all(vals_2d[:,:-1] >= vals_2
```

17d. Fancy indexing — selecting specific rows/columns

In [ ]:
```
A = np.arange(20).reshape(4, 5)
print("A =\n", A)

# select specific rows
print(A[[0, 2]])          # rows 0 and 2 → shape (2,5)

# select specific columns
print(A[:, [1, 3]])       # columns 1 and 3 → shape (4,2)

# select specific (row, col) PAIRS — NOT a submatrix
rows = [0, 1, 2]
cols = [4, 3, 2]
print(A[rows, cols])      # [A[0,4], A[1,3], A[2,2]] → shape (3,)

# for a submatrix from index lists, use np.ix_
print(A[np.ix_([0,2], [1,3])])   # 2x2 submatrix → shape (2,2)
```

17e. Boolean indexing

In [ ]:
```
a = np.array([1., -2., 3., -4., 5.])

# select positive values
print(a[a > 0])            # [1. 3. 5.] — 1D result regardless of input shap

# zero out negatives
b = a.copy()
b[b < 0] = 0.
print(b)

# 2D — boolean mask selects elements, flattens
A = np.array([[1., -2.], [3., -4.]])
print(A[A > 0])            # [1. 3.] — flattened!

# use mask to set values
A[A < 0] = 0.
print(A)
```

17f. Adding dimensions: None / np.newaxis

In [ ]:
```
a = np.array([1., 2., 3.])
print("a.shape:", a.shape)          # (3,)

print(a[:, None].shape)             # (3,1) — column vector
print(a[None, :].shape)             # (1,3) — row vector
print(a[:, np.newaxis].shape)       # same as [:, None]

# useful for broadcasting outer product without einsum
b = np.array([4., 5.])
print((a[:, None] * b[None, :]).shape)   # (3,2) outer product
print(np.allclose(a[:, None] * b, np.outer(a, b)))   # True
```

17g. Ellipsis `...` — index last/first dim of any shape

In [ ]:
```
A = np.random.randn(3, 4, 5, 6)

print(A[0].shape)          # (4,5,6) — same as A[0,:,:,:]
print(A[..., 0].shape)     # (3,4,5) — last dim, same as A[:,:,:,0]
print(A[0, ..., 0].shape)  # (4,5)   — first and last dim fixed

# useful for batched operations
# e.g. residual stream (n_layers, T, d) — get all layer 0 vectors:
resid = np.random.randn(4, 6, 8)
print(resid[0].shape)        # (6,8) — layer 0, all positions, all features
print(resid[:, -1].shape)    # (4,8) — all layers, last position
print(resid[:, -1, :].shape) # (4,8) — same, explicit
```

17h. Diagonal indexing

In [ ]:
```
A = np.arange(9.).reshape(3,3)
print("A =\n", A)

print(np.diag(A))              # [0. 4. 8.] — extract main diagonal
print(np.diag(np.diag(A)))     # diagonal matrix from those values
print(A[range(3), range(3)])   # same as np.diag — fancy index pairs

# off-diagonal
print(np.diag(A, k=1))         # [1. 5.] — one above main diagonal
print(np.diag(A, k=-1))        # [3. 7.] — one below main diagonal

# for induction: extract stripe A[n+i, i+1]
n = 3; T = 2*n
B = np.random.randn(T, T)
stripe = B[np.arange(n,T-1), np.arange(1,n)]   # A[n+i, i+1] for i in 0..n-2
print("stripe shape:", stripe.shape)            # (n-1,)
```

17i. put_along_axis — set values at indices

In [ ]:
```
# put_along_axis: inverse of take_along_axis
A = np.ones((3, 6))
idx = np.array([[0, 2, 4],   # row 0: zero out cols 0,2,4
                [1, 3, 5],   # row 1: zero out cols 1,3,5
                [0, 1, 2]])  # row 2: zero out cols 0,1,2

np.put_along_axis(A, idx, 0., axis=1)
print(A)

# common pattern: zero out non-top-k
B = np.abs(np.random.randn(3, 8))
B_sparse = B.copy()
k = 3
idx_zero = np.argsort(B_sparse, axis=1)[:, :-k]   # indices of BOTTOM (n-k)
np.put_along_axis(B_sparse, idx_zero, 0., axis=1)
print("non-zero per row:", np.sum(B_sparse > 0, axis=1))  # should be [k,k,k
```

17j. Combined — implement from scratch using slicing

In [ ]:
```
# Given A (T, d), implement each in ONE line using slicing only (no loops):

def flip_columns(A):
    # reverse the column order
    raise NotImplementedError()

def last_k_rows(A, k):
    # return last k rows
    raise NotImplementedError()

def every_other_col(A):
    # return columns 0, 2, 4, ... (even indices)
    raise NotImplementedError()

def upper_left(A, k):
    # return top-left k×k submatrix
    raise NotImplementedError()

def anti_diagonal(A):
    # return the anti-diagonal (top-right to bottom-left) of a square matrix
    # hint: flip columns first, then take diagonal
    raise NotImplementedError()
```

In [ ]:
```
A = np.arange(20.).reshape(4, 5)
check("flip_columns",    np.allclose(flip_columns(A), A[:, ::-1]), True)
check("last_k_rows",     np.allclose(last_k_rows(A, 2), A[-2:, :]), True)
check("every_other_col",np.allclose(every_other_col(A), A[:, ::2]), True)

B = np.arange(16.).reshape(4,4)
check("upper_left",      np.allclose(upper_left(B, 2), B[:2, :2]), True)
check("anti_diagonal",   np.allclose(anti_diagonal(B), np.diag(B[:, ::-1])),
```

Solutions — Section 17

In [ ]:
```
def flip_columns(A):        return A[:, ::-1]
def last_k_rows(A, k):      return A[-k:, :]
def every_other_col(A):     return A[:, ::2]
def upper_left(A, k):       return A[:k, :k]
def anti_diagonal(A):       return np.diag(A[:, ::-1])
```